

ADI205/501 Group Project: Live Bilingual Classroom Translation Application

Dr. Zheng Qu

2026-08-27

ADI205/501 Group Project: Live Bilingual Classroom Translation Application

Course: ADI205/501 — AI Augmented Professional Practice

Assigned: August 27, 2026

Due: Saturday, September 12, 2026

Format: Working group application, live demonstration, and technical report

Project Overview

Build an application that helps students follow a live lecture. While the instructor is speaking, the application should capture the speech and display:

1. the original English transcription;
2. corrected text when transcription errors occur; and
3. a translation into Vietnamese or Chinese on a second line.

The purpose is to support comprehension in class and help students improve their English. Your group must demonstrate a working system and explain the engineering decisions behind it.

Group Members

The following groups were announced in class. Names in parentheses are likely roster matches where the student's classroom name was identifiable from the transcript. Please verify the spelling and the final group membership before submission.

Group 1

- Tianlu Sui
- Keyan Luo
- Thien An

Group 2

- Lin
- Ricky
- Jonathan (*Huu Hai Long Ngo*)

Group 3

- Vinh (*Duc Vinh Hoang*)
- Yuhe (*Yuhe Luo*)
- Shimin (*Shiming Wan*)

Group 4

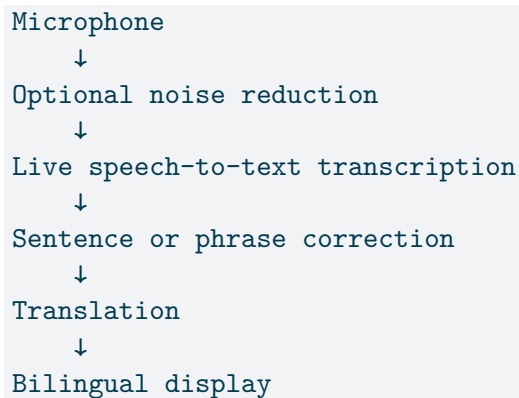
- Jerry (*Jerry Luo*)
- Dang
- Sang

Group 5

- Jerry Chan
- Boyang Liu
- Anh Hui

Required Application Pipeline

Your proposed system should follow this general pipeline:



Correction and translation may be combined into one language-model step if the resulting latency and accuracy are acceptable.

The interface should update incrementally. When a transcription error is identified, the application should correct the relevant word or sentence without unnecessarily disrupting the rest of the live display.

Functional Requirements

The application must:

- accept live microphone input;
- produce a live transcription;
- display the original English text;
- correct transcription errors using a suitable model or service;
- translate the corrected text into Vietnamese or Chinese;
- display the English and translated text clearly, preferably on separate lines;

- operate with approximately **three seconds or less of end-to-end delay**;
- handle realistic classroom noise as well as possible;
- provide a live demonstration; and
- document the system architecture and design decisions.

Some delay is acceptable for a live system, but a delay greater than approximately three seconds may prevent a student from following the lecture.

Engineering Decisions to Address

For each component, identify alternatives, select one option, and justify the choice.

1. Audio capture

Explain whether the application uses:

- a laptop microphone;
- a wireless or external microphone; or
- another input device.

Discuss sound quality, classroom noise, setup requirements, and portability.

2. Noise handling

Explain whether noise reduction is needed and which tool or method is used. Describe how the group will test accuracy in a noisy environment.

3. Transcription

Compare appropriate options, such as:

- a local Whisper-family model;
- another local open-source model;
- a cloud transcription API;
- built-in operating-system transcription; or
- transcription provided by a meeting platform.

Discuss cost, privacy, speed, accuracy, hardware requirements, and internet dependence.

4. Correction

Explain how the system detects or corrects transcription errors. A small local language model may be appropriate when low latency and data privacy are important. A cloud language model may provide stronger correction but introduces network delay, cost, and privacy concerns.

5. Translation

Explain whether translation is performed by the same model used for correction, a separate local model, a platform API, or a built-in translation service.

6. User interface

Explain the interface framework and why it suits the target platform. A native macOS interface may provide a polished and responsive experience. A Python GUI may simplify prototyping or cross-platform support. Compare the selected option with at least one alternative.

7. Media and data formats

If the system also supports recorded audio or video, explain how it handles different formats. A normalization step using `ffmpeg` may be used to inspect media, extract an audio track, and convert it to a format accepted by the transcription engine.

Research Requirements

Use a search-enabled AI tool, such as Perplexity or a search-grounded AI mode, when researching current models, libraries, and APIs. Do not rely solely on a coding assistant’s stored knowledge: software versions and API syntax change.

For each major choice, record:

- the alternatives investigated;
- the current version or implementation considered;
- the advantages and disadvantages;
- the relevant hardware and platform constraints; and
- the reason your group selected the final option.

Your report should make comparisons concrete. Do not write only “we chose this because it is fast.” Explain what it is faster than, under what conditions, and why that difference matters for this application.

Required Deliverables

1. Working application

Submit the source code and any setup instructions needed to run it. Include configuration instructions for models, API keys, microphones, and operating-system permissions where applicable.

2. Live demonstration

Every group must demonstrate the application live. The demonstration should show:

- speech entering through the microphone;
- the original English transcription appearing;
- correction of at least one transcription error, if one occurs or can be demonstrated safely;
- translation into Vietnamese or Chinese; and
- the approximate end-to-end delay.

Test with realistic classroom noise when possible.

3. Technical report

The report must include:

- project objective and intended use case;
- group members;
- system architecture diagram or flowchart;
- pipeline stages;
- options considered for each stage;
- selected tools, models, libraries, and versions;
- reasons for each selection;
- comparison with rejected alternatives;
- privacy, cost, accuracy, speed, and hardware considerations;
- testing procedure and results;
- known limitations and failure cases; and
- instructions for running the application.

4. Evidence of testing

Include screenshots, logs, timing measurements, or a short recorded demonstration showing that the application works. Report latency and describe the conditions under which it was measured.

Suggested Development Sequence

Do not attempt to build the entire application in one step. Develop and test incrementally:

1. Confirm microphone input.
2. Produce a short transcription.
3. Measure transcription delay and accuracy.
4. Add correction for individual sentences.
5. Add translation.
6. Display English and translated text together.
7. Test under classroom noise.
8. Measure total delay from speech to display.
9. Improve the weakest stage.
10. Prepare the demonstration and report.

Assessment Focus

The project will be evaluated primarily on:

- whether the application works in a live demonstration;
- the quality of the system decomposition;
- the justification of technical choices;
- attention to latency and accuracy;
- awareness of privacy, cost, and hardware constraints;
- evidence of testing and honest discussion of limitations; and
- the clarity and completeness of the report.

An application is not complete merely because it runs once. Your group should explain when it works, when it fails, and why the selected architecture is appropriate for the intended classroom use.

Deadline

Saturday, September 12, 2026

Submit the source code, technical report, and demonstration materials as a zip by the deadline. Each group should coordinate its work so that every member understands the architecture and can explain the major design decisions.